

Software Requirements Specification — Crisis-Solving Brain

Software Requirements Specification — General Crisis-Solving Brain

Deep system requirements · React + Python + PostgreSQL · functional, non-functional, architecture, data, API, security & operations · 2026-05-31

This Software Requirements Specification (SRS) defines the functional and non-functional requirements, system and data architecture, interfaces, and security/operations for the General Crisis-Solving Brain, built on **React (frontend) + Python/FastAPI (backend) + PostgreSQL (single primary store with AGE, pgvector, PostGIS, TimescaleDB)**. Requirements are verifiable and grounded in the Zarqa demo case.

1. Purpose, Scope & Stakeholders

1.1 Purpose

This SRS specifies the **General Crisis-Solving Brain** — a domain-agnostic, graph-based, deer-flow-style multi-agent engine that ingests raw signals from ANY case and runs a fixed reasoning loop to surface the **root cause** and a **re-simulated, validated solution**. The engine and agent swarm are invariant; domains plug in as **Domain Packs**. The system's job is to defeat *symptom bias* — to find the causal apex (a pipe rupture) behind the loud, attention-grabbing symptoms (a 911 surge), and to prove a fix works before a human authorizes it.

Core loop (invariant):

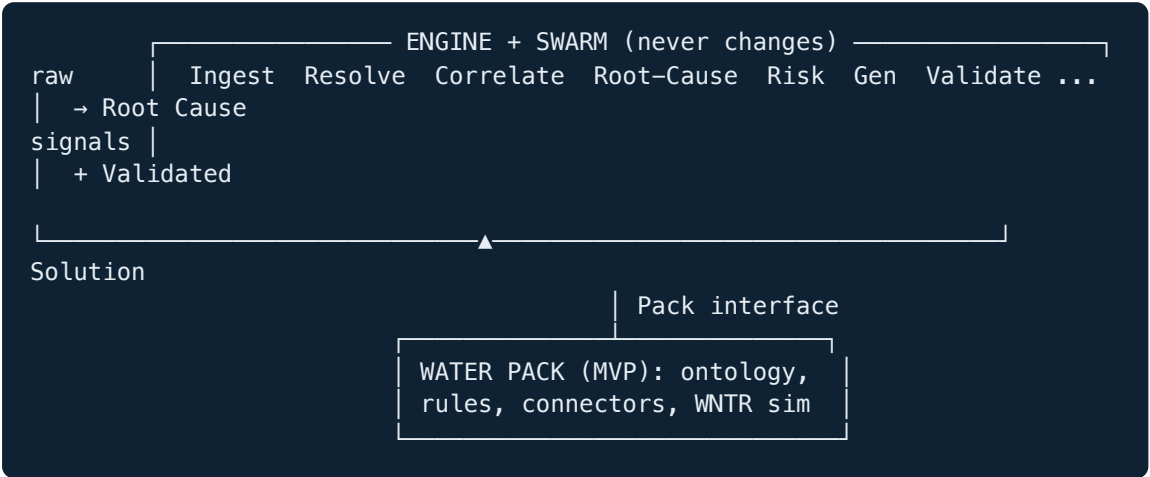
Ingest → Resolve → Correlate → Root-Cause → Risk → Generate-Solution →
Validate → Recommend → Learn

1.2 Scope

In scope (system): the engine (LangGraph swarm, networkx/rustworkx graph core, Splink ER, DoWhy/PyRCA root-cause, PyOD anomaly, OR-Tools optimization), the Pack interface (ontology, propagation rules, connectors, intervention library, simulator adapter), the FastAPI REST+WebSocket API, the PostgreSQL 16 store (AGE property graph, pgvector)

embeddings, PostGIS geo, TimescaleDB signal hypertables), and the React 18 + TypeScript dashboard with its 7-step Wizard.

MVP boundary: ONE Pack — **Water** (WNTR/EPANET adapter) — running the **Zarqa water-pipe cascade** end-to-end: trunk-main rupture PIPE-ZN-44 cascading to hospital strain, traffic, and a **+320% 911 surge**. Success = the brain ranks PIPE-ZN-44 as root cause (NOT the 911/hospital symptoms) and outputs an isolate + bypass + tanker-stopgap fix, **proven by re-simulation** (before/after risk delta). Multi-pack onboarding, full RBAC tenancy, and k8s prod deploy are post-MVP; the architecture is built to accommodate them.



1.3 Intended Users & Stakeholders

Role	Goal	Primary screens	Key permission
Duty Officer	Walk a live case from signals to an authorized fix	Cockpit, Wizard (Steps 1–7), Decision Hub	run case, authorize intervention (human gate)
Analyst	Inspect signals, validate the stitched graph & root cause	Signal Explorer, Incident Graph, Root-Cause Panel	annotate, override correlation, re-run sim
Decision Authority	Approve high-impact interventions	Decision Hub, Solution Review	co-sign / veto at the authorize gate
Admin	Manage users, roles, connectors, observability	Admin console	OAuth2/OIDC + RBAC config
Pack Author	Build/extend a Domain Pack	Pack SDK / repo	register ontology, rules, sim adapter

1.4 Definitions & Acronyms

Term	Meaning
Domain Pack	Pluggable bundle: ontology + propagation rules + connectors + intervention library + simulator adapter
Root Cause / Causal Apex	The upstream node that best explains downstream symptoms (e.g. PIPE-ZN-44)
Signal	Time-stamped raw observation (911 call volume, sensor pressure) → TimescaleDB hypertable
Incident	Stitched graph of correlated entities/events spanning many signals
Intervention	A candidate action (isolate, bypass, dispatch tankers) from the Pack library
Validation	Re-simulation of the fix producing a before/after risk delta
ER	Entity Resolution (Splink/dedupe) — dedupe signals to canonical entities
RCA	Root-Cause Analysis (DoWhy / causal-learn / PyRCA)
AGE / PostGIS / TSDB	Apache AGE (Cypher graph) / PostGIS (geo) / TimescaleDB (time-series), all on PostgreSQL 16

Example signal payload (Zarqa, ingested → TimescaleDB):

```
{ "signal_id": "sig-9f2", "ts": "2026-05-31T08:14:02Z", "source": "psap-911",  
  "entity_ref": "ZONE-ZN", "metric": "call_volume_pct", "value": 320.0,  
  "geo": {"lat": 32.0728, "lon": 36.0876} }
```

1.5 References

- **Blueprint** — General Crisis-Solving Brain (system architecture, deer-flow swarm, Pack model).
- **BUILD BRIEF** — target stack, MVP scope, Zarqa demo case, dashboard Wizard & key screens.
- This SRS: §2 Functional Reqs, §3 Architecture, §4 Data Model, §5 Pack Interface, §6 API, §7 Frontend.

2. Functional Requirements

Each FR is normative ("The system **MUST** ...") and carries one acceptance criterion (AC). IDs are stable; modules map 1:1 to the engine loop and key screens. All examples are grounded in the **Zarqa water-pipe cascade** demo: a **PIPE-ZN-44** trunk-main rupture cascading to hospital strain, traffic gridlock, and a **+320% 911 surge**, whose validated fix is *isolate* + *bypass* + *tanker stopgap*.

Conventions

- **API:** FastAPI REST under `/api/v1/*`; realtime over native WebSocket `/ws/case/{case_id}`.
- **Store:** PostgreSQL 16 with AGE (graph/Cypher), pgvector, PostGIS, TimescaleDB; Redis (queue/pub-sub); MinIO (artifacts).
- **Agents:** LangGraph swarm; nodes emit step events persisted to the case timeline.

2.1 Ingestion & Connectors

ID	The system MUST ...	Acceptance Criterion
FR-1	Expose a connector interface (<code>SourceConnector.poll()/subscribe()</code>) so domain packs register sources (SCADA, 911 CAD, hospital EMR feed, traffic API, social) without engine changes.	Registering the Zarqa pack adds 4 sources; <code>GET /api/v1/sources</code> lists them with <code>pack_id="water.zarqa"</code> and no core code diff.
FR-2	Normalize every inbound signal to a canonical <code>Signal{id, source, ts, geo(Point), type, value, unit, raw}</code> and persist to a TimescaleDB hypertable <code>signals</code> partitioned by <code>ts</code> .	Ingesting <code>{src:"scada", type:"pressure", value:1.2, unit:"bar", geo:[36.08,32.07]}</code> returns 201 and a <code>chunks_time_interval</code> query over <code>signals</code> returns the row in <50 ms.
FR-3	Ingest the demo replay (NDJSON) at ≥ 1000 signals/sec via an Arq worker, computing a pgvector embedding for text/free-form signals.	Replaying the Zarqa trace ($\approx 8k$ signals) completes <10 s; $\geq 99\%$ rows carry a 768-dim <code>embedding</code> .
FR-4	Stream new signals to subscribed clients in real time.	A <code>PIPE-ZN-44</code> pressure-drop signal appears in the Signal Explorer feed within 500 ms of ingest via <code>/ws/case/{id}</code> .
FR-5	Flag anomalies inline using PyOD/river with per-stream baselines.	The +320% 911 call-rate spike is tagged <code>anomaly=true</code> , <code>score$\geq$0.9</code> and surfaced before correlation runs.

2.2 Entity Resolution

ID	The system MUST ...	Acceptance Criterion
FR-6	Resolve raw signal references to canonical <code>Entity</code> records (assets, facilities, zones) using Splink/dedupe with deterministic + probabilistic rules.	"Zarqa New Hospital", "ZN Hosp", and EMR site code <code>J0-ZN-001</code> resolve to one <code>entity_id</code> with match prob ≥ 0.95 .
FR-7	Record every merge as a reversible <code>resolution_edge</code> with score and features, and support unmerge.	<code>POST /entities/{id}/unmerge</code> restores prior IDs and writes an audit row (see FR-37).
FR-8	Geo-anchor entities via PostGIS and map signals to the nearest asset within a configurable radius.	A pressure signal at <code>[36.08, 32.07]</code> binds to <code>PIPE-ZN-44</code> (≤ 25 m) and not to neighboring <code>PIPE-ZN-43</code> .

2.3 Dependency Graph

ID	The system MUST ...	Acceptance Criterion
FR-9	Maintain a property graph in Apache AGE where domain-pack ontology defines node labels and <code>DEPENDS_ON</code> / <code>FEEDS</code> / <code>SERVES</code> edges with propagation weights.	Loading the Zarqa pack creates <code>(:Pipe)-[:FEEDS]->(:Zone)-[:SERVES]->(:Hospital)</code> queryable via Cypher.
FR-10	Answer ancestor/descendant impact queries over the graph.	Cypher <code>MATCH (p:Pipe{id:'PIPE-ZN-44'})-[:FEEDS*1..4]->(n) RETURN n</code> returns the hospital, 2 traffic nodes, and the 911 PSAP node in <100 ms.
FR-11	Support graph versioning so each case snapshots the graph state used for reasoning.	A case stores <code>graph_version</code> and re-opening it renders the identical topology even after the live graph mutates.

```
-- Zarqa propagation seed
MERGE (p:Pipe {id:'PIPE-ZN-44'})
MERGE (z:Zone {id:'ZONE-ZN-N'})
MERGE (h:Hospital {id:'J0-ZN-001'})
MERGE (p)-[:FEEDS {weight:0.9}]->(z)
MERGE (z)-[:SERVES {weight:0.8}]->(h);
```

2.4 Correlation / Stitching

ID	The system MUST ...	Acceptance Criterion
FR-12	Cluster correlated anomalous signals into one <code>Incident</code> using spatio-temporal + graph-distance + embedding similarity.	The pressure drop, hospital admit spike, traffic stall, and 911 surge stitch into a single incident, not four.
FR-13	Render the stitched incident as a React Flow graph payload (<code>nodes</code> , <code>edges</code>) over the API.	<code>GET /incidents/{id}/graph</code> returns ≥ 5 nodes; the canvas (Step 2) draws them with typed edge styling.
FR-14	Continuously re-stitch as new signals arrive, expanding/merging incidents.	Adding a downstream traffic anomaly attaches to the existing incident within one correlation cycle, not a new one.

2.5 Root-Cause

ID	The system MUST ...	Acceptance Criterion
FR-15	Identify the causal apex of an incident using DoWhy/causal-learn/PyRCA over the dependency graph + signal timeline, not signal loudness.	The engine returns <code>root_cause=PIPE-ZN-44</code> and explicitly de-ranks the 911 surge as a <i>symptom</i> .
FR-16	Attach ranked evidence (contributing signals, propagation path, confidence) to each candidate.	Root-Cause Panel (Step 3) shows the apex with confidence ≥ 0.8 and the <code>PIPE→ZONE→HOSPITAL</code> path as evidence.
FR-17	Return a ranked list of ≥ 3 candidate causes with scores so the officer can inspect alternatives.	Response includes the rupture (top) plus ≥ 2 alternates (e.g., valve fault, demand spike) each with a delta score.

2.6 Risk Index

ID	The system MUST ...	Acceptance Criterion
FR-18	Compute a 0–100 Crisis Risk Index per incident from severity $\times$ spread $\times$ population-served $\times$ time-to-cascade.	Zarqa incident scores ≥ 85 ("Critical") on rupture confirmation; value is returned with sub-factor breakdown.
FR-19	Aggregate incident risk into a national index for the Crisis Cockpit and stream updates.	Cockpit gauge moves from baseline to ≥ 85 within 1 s of the rupture incident opening, via WebSocket.
FR-20	Recompute risk after any intervention or simulation and expose before/after deltas.	Post-fix recompute yields ≤ 30 ("Moderate"); delta <code>-55</code> is returned for the Validation screen.

2.7 Solution / Intervention

ID	The system MUST ...	Acceptance Criterion
FR-21	Generate candidate interventions from the domain pack's intervention library targeting the root cause, optionally optimized with OR-Tools.	For <code>PIPE-ZN-44</code> the system proposes <i>isolate valve V-12</i> , <i>open bypass B-3</i> , and <i>deploy 4 tankers</i> , each as a structured <code>Intervention</code> .
FR-22	Score each candidate on projected risk reduction, cost, and time-to-effect, returning a ranked plan.	The combined isolate+bypass+tanker plan ranks #1 with projected risk -55 and ETA shown.
FR-23	Represent a multi-step plan as an ordered, dependency-aware sequence.	Plan enforces <i>isolate before bypass</i> ; reordering returns a 422 validation error.

2.8 Simulation / Validation

ID	The system MUST ...	Acceptance Criterion
FR-24	Run candidate interventions through the pack's simulator adapter (WNTR/EPANET for water; Mesa/PySD generic) as an async job.	Validating the Zarqa plan dispatches a WNTR run via Arq and returns a <code>sim_run_id</code> immediately.
FR-25	Produce a before/after state comparison (risk, affected population, key signals) and persist artifacts to MinIO.	Sim output shows hospital water restored, 911 projected back to baseline; the <code>.inp /results</code> artifact is stored and linkable.
FR-26	Stream simulation progress and final verdict (<code>validated</code> / <code>rejected</code>) to the Simulation Console.	Console (Step 5) shows a progress bar then <code>VALIDATED</code> with the before/after risk chart (visx/Recharts).
FR-27	Block any plan from reaching the Decision Hub until it carries a <code>validated</code> verdict.	Attempting to authorize an unvalidated plan returns 409 with <code>reason="not_validated"</code> .

2.9 Decision Hub & Human Authorization

ID	The system MUST ...	Acceptance Criterion
FR-28	Require an explicit human authorization gate (Step 6) before any intervention is marked actionable; the swarm MUST NOT auto-execute.	LangGraph halts at an <code>interrupt()</code> node; status stays <code>awaiting_authorization</code> until a user acts.
FR-29	Record the decision (<code>approve</code> / <code>reject</code> / <code>modify</code>) with authorizer identity, timestamp, and rationale.	Approving the Zarqa plan writes <code>{actor, role, ts, rationale}</code> ; reject requires a non-empty reason (422 otherwise).
FR-30	Enforce that only an <code>incident_commander</code> / <code>duty_officer</code> role may authorize.	An <code>analyst</code> JWT calling <code>POST /decisions</code> gets 403 (see FR-39).
FR-31	Transition the case to <code>authorized</code> and emit an outcome record for the Learn step on approval.	Authorization flips case state and creates an <code>outcome</code> row capturing the chosen plan and predicted risk.

2.10 Dashboard & Wizard

WIZARD (overlay over screens):

```
[1 Signals]→[2 Stitched Incident]→[3 Root Cause]→[4 Solutions]
→[5 Validation/Sim]→[6 Decide & Authorize]→[7 Outcome/Learn]
▲ guided onboarding + primary case flow; each step deep-links a screen
```

ID	The system MUST ...	Acceptance Criterion
FR-32	Provide the 7-step wizard as the primary case flow and onboarding path, with React 18 + TS, Tailwind, shadcn/ui, TanStack Query (server) + Zustand (UI).	A new user completes Zarqa from Signals to Authorize via the wizard without leaving it; step state survives refresh.
FR-33	Render the key screens — Cockpit, Signal Explorer, Incident Graph (React Flow), Root-Cause Panel, Solution Review, Simulation Console, Decision Hub — each fed by its FR API.	Each screen loads its data contract; Incident Graph draws the FR-13 payload; map layers render via MapLibre GL on OSM tiles.
FR-34	Geolocate signals/entities on a MapLibre GL map with incident overlays.	<code>PIPE-ZN-44</code> and the served hospital render as pinned layers in Zarqa with correct PostGIS coordinates.
FR-35	Reflect live engine progress (each LangGraph node) in the UI in real time and animate transitions (Motion).	As the swarm advances <code>Ingest</code> →...→ <code>Recommend</code> , the wizard step indicator updates live over WebSocket.

2.11 Domain-Pack Management

ID	The system MUST ...	Acceptance Criterion
FR-36	Load a Domain Pack (ontology, propagation rules, connectors, intervention library, simulator adapter) as a versioned, hot-pluggable bundle; the engine/swarm code MUST remain unchanged.	Installing <code>water.zarqa@1.0</code> via <code>POST /packs</code> enables the full demo; a second pack installs alongside without redeploying core.
FR-37	Validate a pack's schema/manifest on install and reject incompatible ones.	A manifest missing <code>simulator_adapter</code> is rejected with a field-level 422 and is not registered.

2.12 Audit / Lineage

ID	The system MUST ...	Acceptance Criterion
FR-38	Persist an append-only, immutable lineage record for every engine step (ingest→learn) linking inputs, outputs, model/agent, and artifacts.	The Zarqa case yields a full chain from raw <code>PIPE-ZN-44</code> signal → root cause → validated plan → authorization, each row hash-chained.
FR-39	Export a complete case audit bundle (decisions, evidence, sim artifacts) to MinIO/S3 on demand.	<code>GET /cases/{id}/audit/export</code> returns a signed bundle reproducing the decision trail; tampering breaks the hash chain check.

2.13 Auth / RBAC

ID	The system MUST ...	Acceptance Criterion
FR-40	Authenticate via OAuth2/OIDC and authorize via RBAC roles (<code>viewer</code> , <code>analyst</code> , <code>duty_officer</code> , <code>incident_commander</code> , <code>admin</code>) enforced on every API and WS route.	An unauthenticated request gets 401; role checks gate FR-30 authorization and pack install (admin-only).
FR-41	Scope WebSocket subscriptions and case access to the user's role/tenant.	A <code>viewer</code> may stream Zarqa signals but receives 403 on <code>POST /decisions</code> ; cross-tenant case IDs are not enumerable.

Traceability: FR-1–5 → Step 1 · FR-12–14 → Step 2 · FR-15–17 → Step 3 · FR-21–23 → Step 4 · FR-24–27 → Step 5 · FR-28–31 → Step 6 · FR-31/38 → Step 7. The Zarqa demo passes only when FR-15 names PIPE-ZN-44 (not the 911 surge) and FR-25 proves the fix by re-simulation.

3. Non-Functional Requirements

NFRs are normative and verifiable. Each carries a **target**, a **measurement method**, and a **gate** (CI, load test, or runbook). All latencies are p95 unless stated; "load" is the Zarqa demo profile: ~1,800 active nodes in the AGE graph, 12 live signal streams, a sustained 250 signals/s burst at the 911 surge (+320% from a 60/min baseline).

3.1 Performance & Latency

#	Metric	Target (p95)	Measurement	Gate
NFR-1	Signal → insight (ingest→root-cause hint)	≤ 5 s, p99 ≤ 9 s	OTel span pipeline.signal_to_insight	k6 load test
NFR-2	AGE Cypher graph query (2-hop neighborhood)	≤ 150 ms	pg_stat_statements , span graph.query	pgbench + EXPLAIN
NFR-3	Full causal apex resolution (PyRCA/DoWhy)	≤ 30 s	LangGraph node timer root_cause.run	nightly bench
NFR-4	Simulation completion (WNTR/EPANET re-sim)	≤ 60 s (Zarqa net)	Arq job duration sim.run	Arq metrics
NFR-5	Dashboard TTI (Cockpit, cold)	≤ 2.5 s on 4G/mid-laptop	Lighthouse CI, web-vitals	CI budget fail
NFR-6	WS signal push (server→client render)	≤ 500 ms	client perf.mark , server emit ts	Playwright trace
NFR-7	React Flow incident graph interaction (pan/zoom, 1.8k nodes)	≥ 50 fps	Chrome trace, long-task < 50 ms	Playwright perf

The signal→insight budget decomposes (sums ≤ 5 s):

```
ingest+validate 300ms | resolve(Splink) 600ms | correlate(AGE) 900ms  
→ anomaly(PyOD) 700ms | rca-hint(PyRCA) 1800ms | persist+emit 700ms
```

3.2 Throughput & Scalability

- **NFR-8 Throughput:** sustain ≥ 300 signals/s ingest with NFR-1 held; absorb a **10× burst (3,000/s, 30 s)** via Redis Streams backpressure, zero loss. Verified by k6 spike profile; queue depth metric `arq.queue.depth` must drain < 60 s.
- **NFR-9 Horizontal scale:** API, Arq workers, and LangGraph executors are stateless and scale by replica count; throughput must rise $\geq 0.8\times$ linearly to 8 replicas. State lives only in PostgreSQL 16 + Redis.
- **NFR-10 Data volume:** TimescaleDB signal hypertable holds ≥ 90 days at 300/s (~2.3 B rows) with NFR-2 intact, using 7-day chunks + native compression after 24 h (target $\geq 10\times$ ratio) + continuous aggregates for Cockpit rollups.
- **NFR-11 Graph scale:** AGE graph to ≥ 1 M nodes / 5 M edges per active national case without breaching NFR-2 (GIN indexes on label/property, materialized 2-hop views for hot subgraphs).

3.3 Availability & Degraded Mode

- **NFR-12 Uptime:** control plane (API, dashboard, WS) $\geq 99.9\%/mo$ (≤ 43 min downtime). Measured by external blackbox probe; SLO error budget tracked in Grafana.
- **NFR-13 Degraded mode:** if RCA/sim engines are down, the system **must still ingest, resolve, correlate, and display the live graph** with a banner "Analysis degraded — root-cause/sim unavailable." Wizard Steps 1–2 stay functional; Steps 3–5 show last-known + staleness age. Verified by chaos test killing the Arq worker pool.
- **NFR-14 RTO/RPO:** RTO ≤ 15 min, RPO ≤ 60 s (PostgreSQL streaming replication + WAL archive to S3/MinIO). DR restore drill runs quarterly.
- **NFR-15 Graceful overload:** beyond NFR-8 burst, shed *enrichment* (embeddings, sim) before *ingest*; never drop a raw signal. HTTP 429 with `Retry-After` on write APIs.

3.4 Reliability & Idempotency

- **NFR-16 Idempotency:** every signal carries a producer `idempotency_key`; duplicate ingest is a no-op. Replaying the Zarqa fixture twice yields **identical** incident graph and apex (`PIPE-ZN-44`).

```
-- ingest dedupe contract
ALTER TABLE signal ADD CONSTRAINT uq_signal_idem UNIQUE (source_id,
    idempotency_key);
-- INSERT ... ON CONFLICT (source_id, idempotency_key) DO NOTHING;
```

- **NFR-17 Exactly-once effects:** LangGraph runs are checkpointed; a crash mid-flow resumes from the last node, never re-executing an *authorized intervention*. Decision Hub authorizations are transactional + append-only.
- **NFR-18 Determinism:** given a fixed input set and seeds, RCA apex and sim verdict are reproducible; CI asserts the Zarqa golden output (root cause = rupture, fix = isolate+bypass+tanker, post-sim 911 load $\Delta \leq$ tolerance).

3.5 Security

#	Control	Requirement
NFR-19	AuthN	OAuth2/OIDC; short-lived JWT (≤ 15 min) + refresh; WS authenticated on connect
NFR-20	AuthZ	RBAC roles <code>viewer</code> / <code>analyst</code> / <code>duty_officer</code> / <code>admin</code> ; only <code>duty_officer</code> + <code>may authorize</code> (Step 6). Enforced server-side per endpoint, not UI-only
NFR-21	Encryption	TLS 1.3 in transit; AES-256 at rest (PostgreSQL volume + S3/MinIO SSE)
NFR-22	Audit	Append-only <code>audit_log</code> (actor, action, before/after, ts) for every authorize/override; tamper-evident hash chain; exportable to S3
NFR-23	Data classification	Tag <code>public</code> / <code>internal</code> / <code>sensitive</code> / <code>restricted</code> ; sensitive geo + PII = restricted
NFR-24	Secrets	No secrets in code/image; injected via env/secret store; gitleaks in CI
NFR-25	Hardening	OWASP ASVS L2; Pydantic v2 validates all inputs; parameterized SQL/Cypher only; SAST + dependency scan gate merges

3.6 Privacy, PII & Data Residency

- **NFR-26 PII minimization:** 911 caller phone/identity is hashed at ingest; the brain reasons over the *strain pattern* (call volume, geo-cluster), never identities. PII columns are restricted-classified, row-level-secured, and excluded from pgvector embeddings.
- **NFR-27 Residency:** all primary data resides in the in-region PostgreSQL 16 cluster (Jordan/EU-config per deployment); no cross-border replication without policy flag. S3/MinIO bucket region pinned.
- **NFR-28 Retention:** raw PII signals purged/anonymized after 30 days; derived aggregates retained per NFR-10. Right-to-erasure job cascades by `subject_id`.

3.7 Observability

- **NFR-29 Tracing:** OpenTelemetry traces span the full loop; every wizard step maps to a named span. A signal is traceable end-to-end by `trace_id` to its incident and recommendation.
- **NFR-30 Metrics:** RED (rate/errors/duration) per service + domain SLIs (NFR-1..7) in Grafana; alerts fire at 80% error budget burn.
- **NFR-31 Logs:** structured JSON to Loki, correlated by `trace_id` / `case_id`; no PII in logs (scrubber enforced).

3.8 Accessibility & UX

- **NFR-32 WCAG 2.1 AA:** keyboard-navigable wizard, $\geq 4.5:1$ contrast, ARIA on React Flow/MapLibre canvases (text alternatives for graph/map state). axe-core in Playwright gates CI; **zero criticals**.

- **NFR-33 Resilience UX:** WS drop shows reconnecting state with last-update age; no silent staleness.

3.9 Maintainability

- **NFR-34 Engine/pack isolation:** the engine + swarm contain **zero domain logic**. A new Domain Pack (ontology, rules, connectors, intervention lib, sim adapter) ships without core edits — enforced by an import-boundary lint (`core/*` may not import `packs/*`).
- **NFR-35 Quality gates:** $\geq 80\%$ coverage on engine/critical paths; typed everywhere (mypy strict, TS strict); Ruff/ESLint clean. Migrations via Alembic only — no manual schema drift.

3.10 Simulation vs Live Isolation (hard boundary)

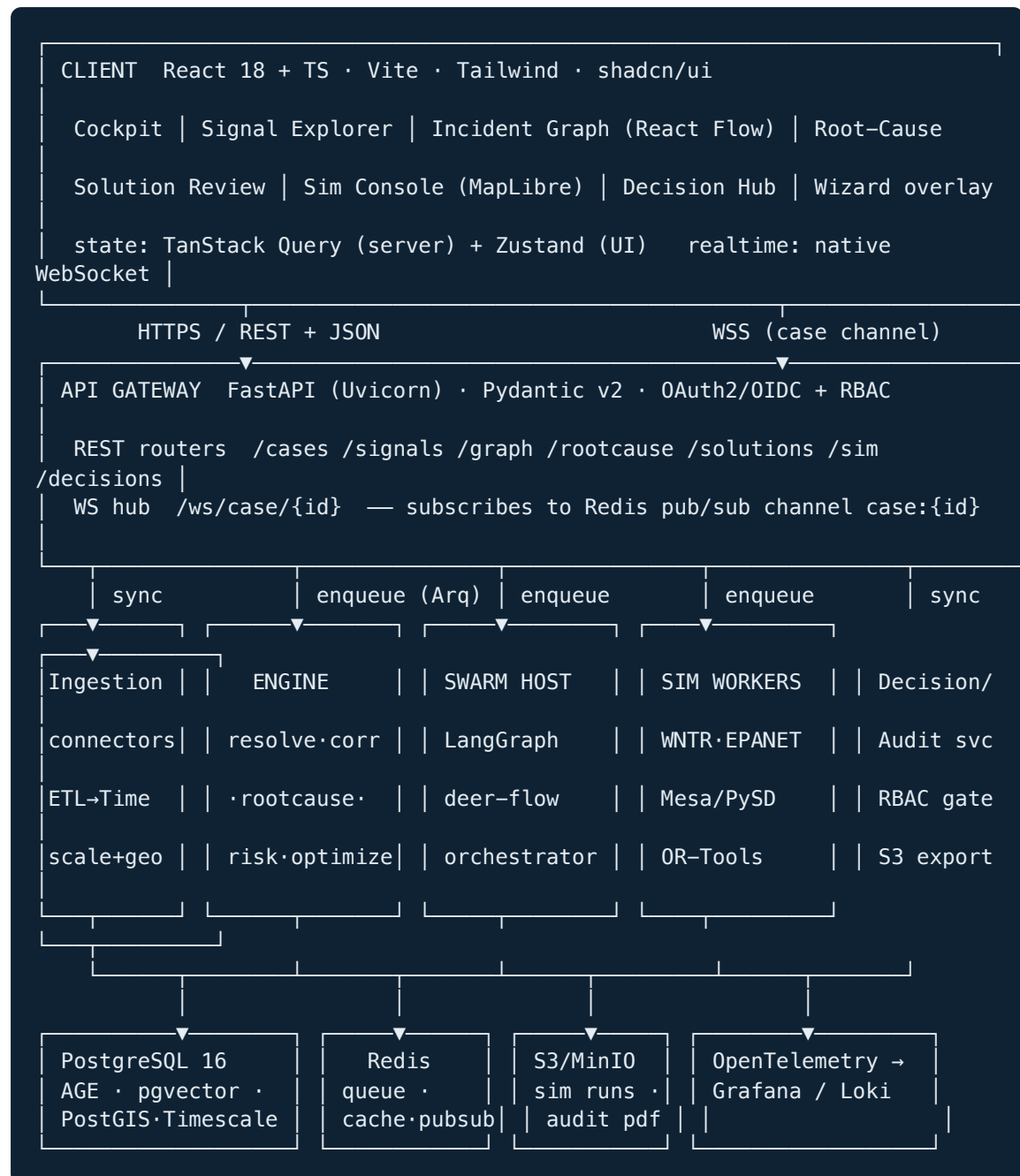
- **NFR-36** Sim and live execute against **separate logical schemas** (`live` vs `sim_<run_id>`); a simulation **can never mutate live graph/signals/state**. Sim writes only to its sandbox + S3 artifact; verdicts are read-back, not applied. Enforced by DB role (sim worker has no `live` write grant) and asserted in CI: a sim run over the Zarqa rupture leaves `live` byte-identical (checksum pre/post).

```
[live schema] --read snapshot--> [sim_<run_id> sandbox] --artifact--> S3
    ^ no write path from sim (role-denied) |
    |_____ verdict read-back only _____|
```

4. System Architecture

The system is a layered, event-driven application: a React/TypeScript client over a FastAPI gateway, a tier of Python application services (ingestion, the domain-agnostic engine, the LangGraph swarm, and simulation workers), and a single PostgreSQL 16 primary store extended with AGE, pgvector, PostGIS, and TimescaleDB, fronted by Redis and an S3/MinIO object store. Synchronous reads (graph fetch, root-cause panel) hit FastAPI directly; long-running work (resolution, root-cause inference, simulation) is dispatched to Arq workers on Redis and streamed back over a WebSocket. The engine and swarm are domain-independent; the Zarqa water case is loaded as a **Domain Pack** (ontology, propagation rules, connectors, intervention library, WNTR/EPANET simulator adapter).

4.1 Layered Diagram



4.2 Component Responsibilities

Component	Stack	Responsibility	Talks to
Client	React 18, React Flow, MapLibre	Wizard + 7 screens; renders graph/map/charts; opens WS per case	Gateway REST + WSS
API Gateway	FastAPI, Pydantic v2	AuthN/Z, request validation, sync reads, job enqueue, WS hub	All services, Redis
Ingestion	Python connectors, SQLAlchemy 2	Normalize raw signals → Timescale hypertable + PostGIS points; emit <code>signal.ingested</code>	Postgres, Redis
Engine	networkx/rustworkx, Splink, DoWhy/PyRCA, PyOD, OR-Tools	Entity resolution, correlation, root-cause (causal apex), risk scoring, intervention optimization	Postgres (AGE), Redis
Swarm Host	LangGraph, Arq worker	Deer-flow orchestrator: routes nodes Ingest→...→Recommend, calls Engine tools, writes step events	Engine, Postgres, Redis
Sim Workers	WNTR/EPANET, Mesa/PySD	Re-simulate before/after intervention; persist hydraulic run + risk delta artifact	Postgres, S3/MinIO
Decision/Audit	FastAPI svc, RBAC	Human authorize gate, immutable decision log, signed audit export	Postgres, S3/MinIO
Data layer	PG16 + AGE/pgvector/PostGIS/Timescale; Redis; MinIO	Graph (Cypher), embeddings (ANN), geo, time-series; queue/cache/pub-sub; artifacts	—

4.3 Sync vs Async, and the Realtime Path

- **Synchronous (FastAPI request→response):** authentication, case CRUD, paginated signal reads, graph snapshot (`MATCH` Cypher via AGE), root-cause panel fetch, decision submission. Target p95 < 300 ms.
- **Asynchronous (Arq workers on Redis):** the engine loop and swarm run, entity resolution over large signal batches, and every simulation. The gateway returns `202 {job_id}` and the result arrives over the WS.
- **Realtime:** each worker step publishes to Redis channel `case:{id}`; the gateway WS hub relays frames to subscribed clients. The Wizard advances steps as frames arrive — no polling.

```
// frame on ws/case/ZQ-2025-0412 as the swarm reaches Root-Cause
{ "type": "node.completed", "step": "root_cause",
  "apex": "PIPE-ZN-44", "label": "trunk-main rupture",
  "confidence": 0.91, "demoted_symptoms": ["911-surge", "hospital-strain"]
}
```

4.4 How the Deer-Flow Swarm Is Hosted

The swarm is a **LangGraph StateGraph** whose nodes are the loop stages; it runs inside an **Arq worker process** (not the web process) so a run never blocks the event loop. A **PostgresSaver** checkpointer persists graph state per `case_id`, making runs resumable and giving the Decision Hub a replayable trace. Nodes are thin: each calls a deterministic **Engine** tool (e.g. `rootcause.infer`, `risk.score`, `solution.optimize`) and emits an event. The LLM plans/explains; the Engine computes — keeping results auditable. Adding a domain means registering a Domain Pack; the graph topology is unchanged.

4.5 Key Sequence — Zarqa Cascade (one case)

```
911 + SCADA + hospital feeds
  | 1. POST /signals (batch)
  ▼
Ingestion → Timescale hypertable + PostGIS geom; publish signal.ingested
  | 2. Gateway enqueues run; returns 202 {job_id}; client opens WS
  ▼
Arq worker boots LangGraph StateGraph(case=ZQ-2025-0412)
  Ingest → Resolve (Splink: dedupe 911 callers, SCADA tags)
          → Correlate (build AGE graph: pipe→pressure→hospital→traffic→911)
          → Root-Cause (DoWhy/PyRCA: apex = PIPE-ZN-44, demote +320% 911
surge)
          → Risk (PyOD + propagation score)
          → Generate-Solution (OR-Tools: isolate + bypass + tanker stopgap)
          → Validate → enqueue Sim worker (WNTR/EPANET re-run)
                        restores pressure, hospital risk 0.88→0.21
                        artifact → MinIO; risk delta → Postgres
          → Recommend (ranked interventions + evidence)
          | each node → Redis case:ZQ-2025-0412 → WS → Wizard advances
          ▼
Decision Hub: duty officer authorizes → immutable log + signed PDF → S3
  ▼
Learn: outcome written back; embeddings (pgvector) updated for recall
```

This path proves the core claim: the brain surfaces the **rupture** as root cause over the louder 911/hospital symptoms, and the re-simulation validates the fix before any human authorization.

5. Data Architecture & PostgreSQL Design

5.1 Why a Single Postgres, Not Polyglot Persistence

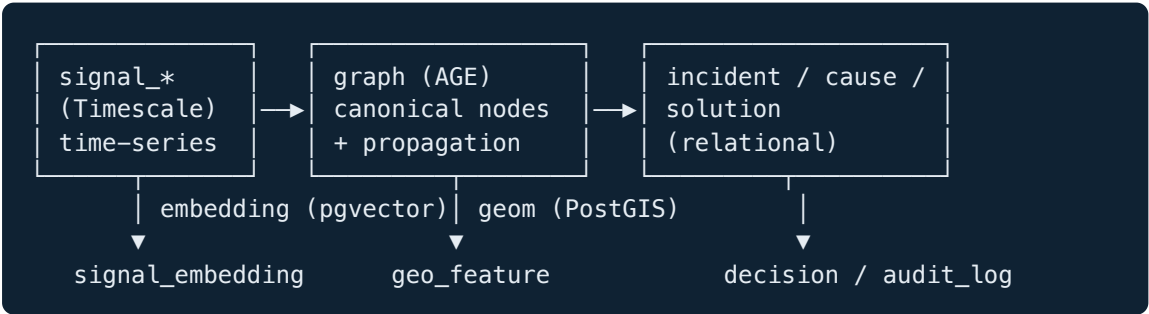
The brain spans five data shapes — time-series signals, a property graph, geo features, vector embeddings, and transactional case records (incidents, causes, solutions, decisions/audit). The naive answer is five stores (InfluxDB + Neo4j + PostGIS + a vector DB + Postgres). We reject that. The crisis loop **Ingest** → **Resolve** → **Correlate** → **Root-Cause** → ... → **Learn** constantly *joins across shapes*: "find signals (time-series) within 2 km (geo) of nodes (graph) whose embeddings (vector) match this incident (relational)." Polyglot persistence makes every such join an application-layer fan-out — N round-trips, no transactional consistency, no single backup, five operational surfaces.

Postgres 16 absorbs all five as extensions in **one ACID engine**:

Shape	Extension	Replaces
Time-series signals	TimescaleDB (hypertables)	InfluxDB
Property graph + Cypher	Apache AGE	Neo4j
Geospatial	PostGIS	standalone PostGIS
Embeddings / ANN	pgvector (ivfflat/HNSW)	Pinecone/Weaviate
Cases/audit	core Postgres	—

One transaction can write a signal, update the causal graph, and append an audit row atomically. One `pg_dump` / PITR backup covers everything. One RBAC model. For a duty-officer dashboard that must show *provably consistent* state under a live cascade, this is the decisive property. We scale read-heavy load with replicas; AGE's graph traversals are bounded (city-scale infrastructure, $\sim 10^4$ – 10^5 nodes), well within Postgres reach. If a single shape ever outgrows Postgres, it can be peeled off later — but not before measured need.

5.2 Schema Domains



- **signals/time-series** — raw + resolved measurements as Timescale hypertables.
- **canonical graph** — entity-resolved nodes (pipes, hospitals, junctions) and typed edges (`FEEDS` , `SERVES` , `CASCADES_TO`) in AGE.
- **incidents/causes/solutions** — the stitched case, the root-cause apex, candidate interventions.

- **decisions/audit** — append-only human-gate record (who authorized what, when, on which evidence).
- **embeddings** — pgvector columns for signal/text dedup and similar-case recall.
- **geo** — PostGIS geometries for the MapLibre layer and spatial correlation.

5.3 Representative DDL (Zarqa-grounded)

```
CREATE EXTENSION IF NOT EXISTS timescaledb;
CREATE EXTENSION IF NOT EXISTS age;
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS vector;
LOAD 'age'; SET search_path = ag_catalog, public;

-- 5.3.1 Signals: Timescale hypertable, provenance-isolated
CREATE TABLE signal (
  id          BIGINT GENERATED ALWAYS AS IDENTITY,
  ts          TIMESTAMPTZ      NOT NULL,
  source      TEXT             NOT NULL,          -- 'scada','911','traffic'
  metric      TEXT             NOT NULL,          --
  value       DOUBLE PRECISION,
  node_ref    TEXT,              -- canonical id, e.g.
  geom        GEOMETRY(Point, 4326),
  provenance  TEXT             NOT NULL DEFAULT 'live'
  CHECK (provenance IN ('live','sim')),
  sim_run_id  UUID,              -- NULL when live
  PRIMARY KEY (id, ts)
);
SELECT create_hypertable('signal','ts', chunk_time_interval => INTERVAL '1
day');
ALTER TABLE signal SET (timescaledb.compress,
  timescaledb.compress_segmentby = 'source,metric,provenance');
SELECT add_compression_policy('signal', INTERVAL '7 days');
SELECT add_retention_policy ('signal', INTERVAL '180 days'); -- live raw
-- continuous aggregate for the cockpit risk index
CREATE MATERIALIZED VIEW signal_1m WITH (timescaledb.continuous) AS
SELECT time_bucket('1 minute', ts) bucket, source, metric, node_ref,
  avg(value) avg_v, max(value) max_v, count(*) n
FROM signal WHERE provenance='live' GROUP BY 1,2,3,4;

-- 5.3.2 Embeddings (pgvector) for dedup + similar-case recall
CREATE TABLE signal_embedding (
  signal_id BIGINT, ts TIMESTAMPTZ,
  embedding vector(384) NOT NULL,
  PRIMARY KEY (signal_id, ts)
);

-- 5.3.3 Incident / Cause / Solution (relational core)
CREATE TABLE incident (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title TEXT NOT NULL, status TEXT NOT NULL DEFAULT 'open',
  opened_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  centroid GEOMETRY(Point,4326),
  embedding vector(768) -- case-similarity recall
);
CREATE TABLE root_cause (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_id UUID REFERENCES incident(id),
  apex_node_ref TEXT NOT NULL, -- 'PIPE-ZN-44'
  confidence NUMERIC(4,3),
  method TEXT, -- 'pyrca','dowhy'
  evidence JSONB NOT NULL -- paths, scores
);
```

```

CREATE TABLE solution (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_id UUID REFERENCES incident(id),
  interventions JSONB NOT NULL,          --
    ['isolate','bypass','tanker']
  sim_run_id UUID, risk_before NUMERIC, risk_after NUMERIC,
  validated BOOLEAN DEFAULT false
);

-- 5.3.4 Decisions / append-only audit
CREATE TABLE decision (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  solution_id UUID REFERENCES solution(id),
  actor TEXT NOT NULL, action TEXT NOT NULL,    -- 'authorize','reject'
  decided_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  rationale TEXT, evidence_snapshot JSONB NOT NULL
);

CREATE TABLE audit_log (                -- immutable, hash-chained
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  ts TIMESTAMPTZ NOT NULL DEFAULT now(),
  actor TEXT, entity TEXT, entity_id UUID,
  payload JSONB NOT NULL, prev_hash BYTEA, row_hash BYTEA
);

-- 5.3.5 Canonical graph in AGE
SELECT create_graph('crisis');
SELECT * FROM cypher('crisis', $$
  CREATE (:Pipe {ref:'PIPE-ZN-44', diameter_mm:600})
    -[:FEEDS]->(:Zone {ref:'ZN-44'})
    -[:SERVES]->(:Hospital {ref:'HOSP-ZARQA-1'})
  $$) AS (v agtype);

```

5.4 Indexing Strategy

Access pattern	Index	Column
Time + tag scans	Timescale chunk + BTREE	signal(node_ref, ts)
Geo radius / ST_DWithin	GiST	signal.geom, incident.centroid, geo_feature.geom
JSONB evidence / payload search	GIN (jsonb_path_ops)	root_cause.evidence, audit_log.payload
Full-text on signal text	GIN (tsvector)	raw 911 text
Embedding ANN	HNSW (cosine)	incident.embedding, signal_embedding.embedding

```

CREATE INDEX ON signal USING gist (geom);
CREATE INDEX ON root_cause USING gin (evidence jsonb_path_ops);
CREATE INDEX ON incident USING hnsw (embedding vector_cosine_ops)
  WITH (m=16, ef_construction=64);

```

We choose **HNSW over ivfflat**: better recall at low latency and no retrain-on-drift, at higher build cost — acceptable for our modest vector counts. ivfflat stays the fallback if memory pressure appears.

5.5 Graph Storage: AGE vs Adjacency Table

	Apache AGE (chosen)	Plain adjacency table
Multi-hop traversal	native Cypher, readable	recursive CTEs, verbose
Variable-length paths (cascade tracing)	<code>-[:CASCADES_T0*1..5]-></code>	hard, slow
Joins to relational/geo	same DB, one txn	same DB, one txn
Tooling maturity	younger, fewer ops eyes	rock-solid SQL

Root-cause tracing is inherently *variable-depth path search* over the propagation graph — exactly AGE's strength and recursive-CTE's weakness. We accept AGE's relative immaturity because the alternative bloats the hottest query in the loop. Critically, AGE lives **inside** the same Postgres, so a Cypher cascade query and a PostGIS radius filter run in one transaction.

5.6 Sim/Live Provenance Isolation

Re-simulation must never pollute live state. Isolation is enforced at the data layer, not by convention:

- Every signal/solution row carries `provenance ∈ {live,sim}` + a `sim_run_id`; the `CHECK` constraint makes "sim" rows first-class but separable.
- Continuous aggregates and the cockpit risk index filter `provenance='live'`.
- AGE simulation writes go to a **separate graph** (`crisis_sim_<run>`), torn down after validation; the canonical `crisis` graph is read-only during sims.
- Sim artifacts (WNTR/EPANET outputs) land in S3/MinIO keyed by `sim_run_id`; only the before/after risk scalars are promoted into `solution`.

This lets Step 5 (Validation) run the Zarqa fix — isolate `PIPE-ZN-44` + bypass + tanker — against a graph copy, compare `risk_before / risk_after`, and surface the result without touching the live feed.

5.7 Migrations, Retention, Backup

- **Migrations:** Alembic. Extension installs and AGE/Timescale setup live in an early baseline migration; `op.execute()` carries raw DDL Alembic can't autogenerate (hypertables, `create_graph`, vector indexes). Each domain pack ships additive migrations only.
 - **Retention:** raw live signals 180 d → after which the 1-minute continuous aggregate is the system of record; audit/decision tables **never** expire.
 - **Backup:** nightly `pg_dump` plus continuous WAL archiving for PITR; one logical/physical backup covers graph, geo, vectors, and time-series together — the core payoff of single-store. S3/MinIO sim artifacts are lifecycle-expired independently.
-

6. Interface & API Requirements

The platform exposes one HTTP/JSON REST surface and one WebSocket channel, both served by FastAPI behind a single OpenAPI 3.1 schema (`/openapi.json` , Swagger at `/docs`). All resources are case-scoped; the canonical endpoint is a **Case** (the Zarqa cascade is `case_8f2a`).

6.1 REST Resource Model & Conventions

Base path `/api/v1` . Resource-oriented, plural nouns, sub-resources nested under their case. JSON bodies are Pydantic v2 models; `snake_case` fields; RFC 3339 UTC timestamps; UUIDv7 IDs prefixed by type (`case_` , `sig_` , `inc_` , `rc_` , `sol_` , `sim_`).

POST	<code>/cases</code>	create case (ingest config + domain_pack)
GET	<code>/cases?status=active&page[...]</code>	list (cursor paginated)
GET	<code>/cases/{id}</code>	case envelope + current wizard step
GET	<code>/cases/{id}/signals</code>	Timescale+pgvector backed feed
POST	<code>/cases/{id}/signals</code>	push raw signal (adapter ingress)
GET	<code>/cases/{id}/incident</code>	stitched graph (AGE/Cypher projection)
POST	<code>/cases/{id}/correlate</code>	trigger resolve+correlate stage
GET	<code>/cases/{id}/root-cause</code>	causal apex + ranked evidence
POST	<code>/cases/{id}/root-cause:analyze</code>	run RCA (DoWhy/PyRCA)
GET	<code>/cases/{id}/solutions</code>	candidate interventions
POST	<code>/cases/{id}/solutions:generate</code>	OR-Tools generation
POST	<code>/cases/{id}/simulations</code>	run WNTR/EPANET before/after sim
GET	<code>/cases/{id}/simulations/{sid}</code>	sim run + risk delta + S3 artifact URL
POST	<code>/cases/{id}/decisions</code>	authorize/reject (human gate)
GET	<code>/cases/{id}/audit</code>	immutable event log

Non-CRUD verbs use the `:action` colon-suffix convention (`:analyze` , `:generate`) to keep them distinct from sub-resource collections. Field selection via `?fields=` , graph depth via `?depth=` .

Pagination — cursor-based (keyset over (`created_at` , `id`)); never offset, since signal feeds are append-heavy. Response: `{ "data": [...], "page": { "next_cursor": "...", "has_more": true, "limit": 50 } }` . Default limit 50, max 200.

Versioning — URL major version (`/api/v1`). Additive changes are non-breaking; breaking changes bump to `/api/v2` and the old version is supported ≥ 6 months. Per-response `X-API-Version` header; deprecations announced via `Sunset` + `Deprecation` headers.

Idempotency — all unsafe POSTs (especially `:generate` , `simulations` , `decisions`) require an `Idempotency-Key` header (client UUID). Keys + response hash are stored in Redis (24h TTL); a replay returns the original `201` with header `Idempotency-Replayed: true` . This prevents a duty officer double-authorizing the same isolation order on `PIPE-ZN-44` .

6.2 Error Taxonomy

RFC 9457 application/problem+json . Stable type URN, machine code , human detail , plus trace_id (OpenTelemetry) for Grafana/Loki correlation.

```
{ "type": "https://errors.crisis.io/validation",
  "code": "SIGNAL_SCHEMA_INVALID", "status": 422,
  "detail": "field 'geo' is not valid PostGIS GeoJSON",
  "trace_id": "0af7651916cd43dd8448eb211c80319c",
  "errors": [{ "field": "geo.coordinates", "issue": "lat out of range" }] }
```

HTTP	code prefix	Meaning
400	BAD_REQUEST_*	malformed request
401	AUTH_*	missing/expired JWT
403	RBAC_SCOPE_DENIED	valid token, insufficient scope
404	*_NOT_FOUND	unknown resource
409	STATE_CONFLICT	wrong wizard step / version clash
422	*_SCHEMA_INVALID	Pydantic validation failure
429	RATE_LIMITED	quota exceeded (Retry-After)
503	ENGINE_UNAVAILABLE	LangGraph/sim worker down

6.3 WebSocket Event Contract

wss://.../api/v1/ws/cases/{id}?token={jwt} — server→client push of LangGraph node transitions and live signals. Subprotocol JSON; envelope: { "event": "...", "case_id": "...", "seq": 42, "ts": "...", "data": {...} }. Monotonic seq lets the React/TanStack client detect gaps and replay via GET /cases/{id}/events?after_seq= .

event	payload
signal.ingested	raw signal (drives Signal Explorer)
incident.updated	added/changed graph nodes+edges (React Flow)
stage.transition	{from,to,node} — advances wizard
rootcause.found	{node:"PIPE-ZN-44", confidence:0.92}
solution.proposed	candidate intervention
simulation.progress	{sim_id, pct, eta_s}
simulation.complete	before/after risk index
decision.recorded	authorization + actor
error	problem+json mirror

Client→server: `{"op":"subscribe","topics":["signals","graph"]}`, ping / pong heartbeat (30s; idle close 60s). Auth re-validated on connect; expired JWT → close code 4401 . Redis pub/sub fans events across API replicas.

6.4 Authentication & RBAC

OAuth2/OIDC (Authorization Code + PKCE) via the org IdP. Access token = JWT (RS256, 15-min TTL) with claims `sub`, `roles`, `scopes`, `aud`, `exp`; refresh token (rotating, 8h, stored httpOnly) at `POST /auth/refresh`. FastAPI dependency validates signature (JWKS cache), audience, expiry, then enforces a required scope per route. Scopes are `resource:action`; roles bundle scopes.

Roles: **viewer** (read), **analyst** (run engine stages), **duty_officer** (authorize), **admin** (packs/users).

Endpoint	Method	Required scope	Roles
<code>/cases</code>	POST	<code>case:create</code>	analyst, duty_officer, admin
<code>/cases/{id}</code>	GET	<code>case:read</code>	all
<code>/cases/{id}/signals</code>	POST	<code>signal:ingest</code>	adapter (svc), analyst
<code>/cases/{id}/correlate</code>	POST	<code>engine:run</code>	analyst, duty_officer
<code>/cases/{id}/root-cause:analyze</code>	POST	<code>engine:run</code>	analyst, duty_officer
<code>/cases/{id}/solutions:generate</code>	POST	<code>solution:generate</code>	analyst, duty_officer
<code>/cases/{id}/simulations</code>	POST	<code>simulation:run</code>	analyst, duty_officer
<code>/cases/{id}/decisions</code>	POST	<code>decision:authorize</code>	duty_officer, admin
<code>/cases/{id}/audit</code>	GET	<code>audit:read</code>	duty_officer, admin
<code>/admin/domain-packs</code>	POST	<code>pack:manage</code>	admin

Service-to-service (signal adapters) uses the OAuth2 client-credentials grant → a service principal carrying only `signal:ingest`. Every mutating call writes an append-only `audit` row (actor, scope, idempotency key, before/after hash).

6.5 Rate Limiting

Redis sliding-window per (principal, route-class). Headers on every response: `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset`. Defaults: read 600/min; engine actions (`:analyze`, `:generate`, `simulations`) 20/min (compute-heavy);

`signal:ingest` 5,000/min/adaptor to absorb the +320% 911 surge. Breach → 429 +
Retry-After .

6.6 External Interfaces

Signal-source adapters — push to `POST /cases/{id}/signals` or stream into the ingest queue (Arq/Redis). Each adapter declares a manifest (source type, schema, mapping to domain ontology). Canonical envelope:

```
{ "source":"jordan_911_cad", "type":"call_volume",  
  "entity_ref":"ZONE-ZARQA-N", "value":420, "unit":"pct_baseline",  
  "geo":{"type":"Point","coordinates":[36.09,32.07]},  
  "observed_at":"2026-05-31T08:14:00Z", "embedding_text":"surge of water-  
    outage calls" }
```

Ingest pipeline: validate → PostGIS geo + Timescale hypertable insert → pgvector
embedding → entity resolution (Splink) → emit `signal.ingested` .

Domain-pack plugin contract — packs are versioned bundles registered via
`/admin/domain-packs` , implementing five interfaces: `Ontology` , `PropagationRules` ,
`Connectors` , `InterventionLibrary` , `SimulatorAdapter` (the water pack wraps
WNTR/EPANET). The engine + LangGraph swarm are pack-agnostic;
`case.domain_pack="water@1.3"` selects bindings at runtime.

Export/Audit — `GET /cases/{id}/audit` (paginated JSON) and `POST`
`/cases/{id}/exports` produce a signed, hash-chained bundle (case timeline, RCA
evidence, sim artifacts) to S3/MinIO, returned as a time-limited presigned URL for regulator
handoff.

7. Security, Compliance, Deployment & Operations

This section specifies non-functional requirements for running the crisis-solving brain in a
national operations context. The brain authorizes physical-world interventions (isolating
`PIPE-ZN-44` , dispatching tankers), so security and auditability are first-class, not bolt-ons.

7.1 AuthN / AuthZ / RBAC

SR-SEC-1 — Authentication MUST use OAuth2/OIDC (Authorization Code + PKCE) against
the agency IdP (Keycloak in dev). FastAPI validates RS256 JWTs via JWKS; access tokens
≤15 min, refresh rotated. WebSocket connections authenticate via a short-lived ticket issued
over HTTPS, never a query-string token.

SR-SEC-2 — Authorization is RBAC enforced at the API boundary (FastAPI dependency
`require perm`) and re-checked in the data layer. Roles:

Role	Reads	Acts
viewer	cockpit, signals, graph	—
analyst	+ sim console, root-cause	run sims, annotate
duty_officer	all	advance wizard, request authorization
commander	all	authorize interventions (Step 6 gate)
domain_admin	pack config	edit Domain Pack (ontology, intervention library)
platform_admin	system	manage users, secrets, deploys

SR-SEC-3 — Authorization decisions (isolate/bypass) require the `commander` role **and** step-up re-authentication (WebAuthn/MFA) within the request. The human gate at Wizard Step 6 **MUST NOT** be satisfiable by `duty_officer` alone — separation of duties between *recommend* and *authorize*.

7.2 Data Classification, Encryption & Privacy

SR-SEC-4 — Data is classified and handled per row/column:

Class	Example (Zarqa)	At Rest	In Transit
PUBLIC	pipe network topology	PG TDE	TLS 1.3
INTERNAL	sim runs, risk index	PG TDE	TLS 1.3
SENSITIVE	911 call records, caller geo	TDE + app-layer (pgcrypto)	TLS 1.3
PII	hospital patient counts, reporter phone	column-encrypt + masked views	TLS 1.3 + mTLS service-to-service

SR-SEC-5 — At rest: PostgreSQL 16 on encrypted volumes (LUKS / cloud KMS-backed EBS); SENSITIVE/PII columns additionally encrypted via pgcrypto with keys from the secrets manager. S3/MinIO artifacts use SSE-KMS. Redis is `requirepass` + TLS; no PII persisted to Redis.

SR-SEC-6 — In transit: TLS 1.3 terminates at the ingress; internal mesh (FastAPI ↔ Celery/Arq workers ↔ LangGraph ↔ PG/AGE/pgvector/PostGIS/Timescale) uses mTLS. Embeddings written to pgvector **MUST** be derived from de-identified text (caller PII stripped before vectorization).

SR-SEC-7 — Privacy/residency: all PII and signal time-series remain in the in-country region; the single PostgreSQL primary and its read replicas are region-pinned. PII retention ≤90 days with automated TimescaleDB drop-chunk policies; right-to-erasure resolves through the entity-resolution layer (Splink) so deleting a person purges all merged identities.

7.3 Audit Trail & Decision Lineage

SR-SEC-8 — Every state-changing action and every agent decision **MUST** be recorded in an append-only, hash-chained audit log (each row stores `prev_hash` ; tamper-evident). This is

the system of record for *why* the brain blamed the rupture and authorized the fix.

```
CREATE TABLE audit_log (  
  id          BIGSERIAL PRIMARY KEY,  
  ts          TIMESTAMPTZ NOT NULL DEFAULT now(),  
  actor       TEXT NOT NULL,           -- user sub OR agent node id  
  actor_kind  TEXT NOT NULL,           -- 'human' | 'agent'  
  case_id     TEXT NOT NULL,           -- e.g. 'ZARQA-2026-05-31'  
  action      TEXT NOT NULL,           --  
              'root_cause.assert','intervention.authorize'  
  payload     JSONB NOT NULL,          -- inputs, evidence refs, model+pack  
              version  
  prev_hash   BYTEA,  
  row_hash    BYTEA NOT NULL          --  
              sha256(prev_hash||ts||actor||action||payload)  
);
```

SR-SEC-9 — Decision lineage MUST be reconstructable: each LangGraph node logs its inputs, the graph/Domain-Pack version, and evidence node IDs (AGE) that justified the output. A duty officer MUST be able to click the Zarqa root cause and trace: 911 surge → hospital strain → low-pressure zone → PIPE-ZN-44 rupture, with the PyRCA score and the simulation artifact (S3 URI) that validated isolate+bypass+tanker.

SR-SEC-10 — Audit exports (signed JSONL to S3) MUST be available per case for after-action review; logs are WORM-retained ≥7 years; reads are themselves audited.

7.4 Secrets & Key Management

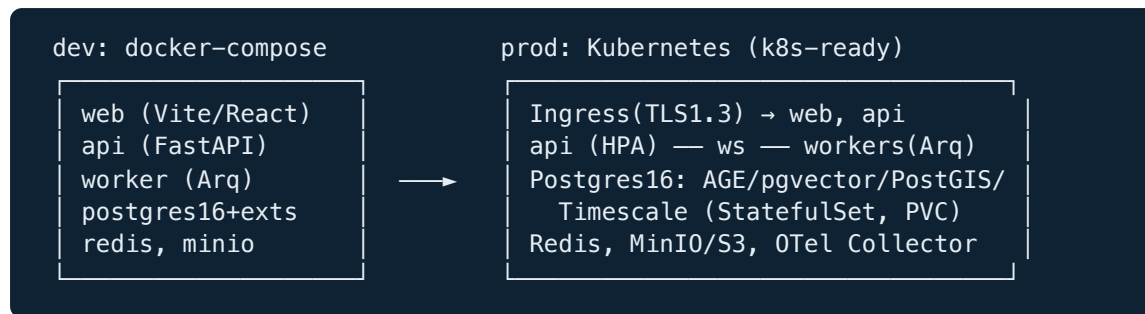
SR-SEC-11 — No secrets in images, repos, or env files committed to git. Secrets resolve at runtime from a manager (HashiCorp Vault / cloud secrets manager); dev uses a git-ignored .env + docker-compose secrets. Keys (JWT signing, pgcrypto, KMS data keys) rotate on schedule (signing ≤90 days) with envelope encryption; rotation is zero-downtime via key versioning. CI uses short-lived OIDC tokens, never long-lived cloud keys.

7.5 Input-Trust & Adversarial Hardening

SR-SEC-12 — All ingested signals are **untrusted**. The Ingest node MUST validate against Pydantic v2 schemas, enforce per-source rate limits, and quarantine malformed/anomalous batches (PyOD) before they reach the graph. A spoofed +320% 911 surge MUST NOT alone drive an authorization — the brain weights corroborating multi-source evidence and flags single-source spikes.

SR-SEC-13 — LLM/agent trust boundary: any free-text routed to LangGraph agents is treated as data, not instructions (prompt-injection defense). Tool calls the swarm may invoke are allow-listed; the swarm CAN recommend but CANNOT authorize or execute physical interventions — execution is gated behind commander + Step 6. All DB access from request handlers uses parameterized SQLAlchemy 2 / Cypher; no string-built queries.

7.6 Deployment & Infrastructure



SR-OPS-1 — Dev runs via `docker-compose up` (one-command bring-up incl. seeded Zarqa case). Prod is Kubernetes-ready: stateless `web / api / worker` Deployments with HPA; PostgreSQL as a StatefulSet (or managed PG with the four extensions) with PVCs; readiness/liveness probes; resource limits; PodSecurity `restricted`; NetworkPolicies restricting DB access to `api/worker` only.

SR-OPS-2 — Config is 12-factor via env vars, validated at boot by a Pydantic `Settings`; missing/invalid config fails fast. Environments: `dev`, `staging`, `prod`, each with isolated DB, secrets, and Domain-Pack versions.

SR-OPS-3 — CI/CD on GitHub Actions: lint (ruff/eslint) → typecheck (mypy/tsc) → unit+integration (pytest, Vitest) → build → scan (Trivy image, `pip-audit / npm audit`, CodeQL, Gitleaks) → push to registry → deploy staging → **Zarqa acceptance suite (Playwright)** → manual approval → prod. Images are SBOM-tagged and signed (cosign).

SR-OPS-4 — Observability: OpenTelemetry traces/metrics/logs from FastAPI, workers, and LangGraph nodes to an OTel Collector → Grafana/Loki/Tempo. A single `case_id + trace_id` correlates UI action → agent loop → SQL/Cypher → sim run. Alerts: ingest lag, sim-queue depth, auth failures, audit-chain break.

SR-OPS-5 — Backup/DR: nightly base backups + continuous PITR (WAL archive to S3); MinIO artifacts versioned + replicated. RPO ≤ 5 min, RTO ≤ 30 min; restore drills quarterly and verified by replaying the Zarqa case post-restore.

7.7 Test & Quality Strategy

Layer	Tooling	Coverage gate
Unit	pytest, Vitest	≥85% engine/agent nodes
Integration	pytest + ephemeral PG (AGE/pgvector/PostGIS/Timescale), testcontainers	resolver, correlation, PyRCA, sim adapter
Contract	schemathesis (OpenAPI), WS message schemas	REST + realtime
E2E	Playwright	full wizard Steps 1→7
Load	Locust	911-surge burst ingest
Security	CodeQL, Trivy, ZAP baseline	no high/critical

SR-QA-1 (Zarqa Acceptance Suite) — A blocking E2E suite seeds the Zarqa cascade and asserts the brain: (a) ingests the +320% 911 surge + hospital + traffic signals; (b) stitches one incident graph; (c) **identifies PIPE-ZN-44 rupture as root cause, NOT the 911/hospital symptoms** (asserted on the causal apex + PyRCA score, with symptom nodes ranked below); (d) generates the `isolate + bypass + tanker` solution; (e) re-simulation shows risk/pressure recovery below threshold (before/after delta asserted); (f) the `commander` gate blocks lower roles; (g) audit chain verifies and lineage is reconstructable. Any failure fails the pipeline.
